

Object Oriented Programming

Principle of LSP

IT560



Sourish Dasgupta

Assistant Professor, DA-IICT, Gandhinagar

<https://daiict.ac.in/faculty/sourish-dasgupta>

Why the “Base (Abstract) Class” Animal?

```
class Animal:
    def speak(self):
        raise NotImplementedError("Subclasses must implement this method")

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"
```

`animal = Animal()`

`animal.speak()`

Adheres to design principles like the **Open-Closed Principle (OCP)** and **Liskov Substitution Principle (LSP)**

1. **Common interface (contract)** (`speak`) that ensures consistency
2. Enable **polymorphism**, allowing the client code to work with animals **generically**
3. Improve **extensibility**, making it easy to **add new** animals
4. Maintain a **clear and organized structure**

What if we remove Base Class?

```
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def sound(self): # Wrong method name
        return "Meow!"

# Client Code
def animal_sound(animal):
    return animal.speak() # Runtime error for Cat
```

Removing the `Animal` class would lead to a less maintainable, error-prone, and less scalable system

A strictly Abstract Class in Python

```
from abc import ABC, abstractmethod

# Abstract Animal class
class Animal(ABC):
    @abstractmethod
    def speak(self):
        pass # Abstract method that must be implemented by subclasses

animal = Animal()
```

Liskov Substitution Principle (LSP)

"Objects of a superclass should be **replaceable** with objects of its subclasses **without altering the correctness of the program**"

- Any instance of a subclass should be able to replace an instance of its superclass without breaking the behavior expected by the client code
- Subclasses must **fully honor the contract (interface and behavior)** defined by the parent class



LSP in Action

```
class Animal:
    def speak(self):
        raise NotImplementedError("Subclasses must implement this method")
```

```
class Dog(Animal):
    def speak(self):
        return "Woof!"
```

```
class Cat(Animal):
    def speak(self):
```

```
def animal_sound(animal_type):
    animal = AnimalFactory.create_animal(animal_type)
    return animal.speak()
```

- The client code (`animal_sound`) interacts with objects through the `Animal` type.
- Doesn't need to know whether it's working with a `Dog`, `Cat`, or any other subclass of `Animal`.
- As long as each subclass of `Animal` properly implements the `speak` method, the client code will work correctly

LSP in Action: Substitution w/o Breaking Behavior

```
animal = Dog() # Substitute a Dog for Animal
print(animal.speak()) # Output: Woof!

animal = Cat() # Substitute a Cat for Animal
print(animal.speak()) # Output: Meow!
```

- A **Dog** or **Cat** object can replace an **Animal** object in the **animal_sound** function *without requiring changes to the function's logic or causing errors*

Consistency of the Interface (**speak**):

- The **Animal** class defines the **speak** method as a contract.
- Every subclass (**Dog**, **Cat**) must implement **speak** with behavior that aligns with the expectations of the client code.

LSP in Action: Extensibility w/o Violation

```
class Bird(Animal):  
    def speak(self):  
        return "Tweet!"  
  
# Usage:  
bird = Bird()  
print(bird.speak()) # Output: Tweet!
```

Adding a new animal (e.g., `Bird`) still adheres to the LSP

What violates LSP?

Breaking LSP: Violation of Expectation

```
class Dog(Animal):  
    def speak(self):  
        return 42 # Violates the expected return type (string)  
  
# Client Code:  
print(Animal.sound("dog")) # Breaks expectations, returns 42 instead of "Woof!"
```

Breaking LSP: Unexpected Modifications

```
class Cat(Animal):  
    def speak(self, volume): # Incompatible method signature  
        return "Meow!" * volume
```

Breaking LSP: Unexpected Exceptions

```
class Bird(Animal):  
    def speak(self):  
        raise NotImplementedError("Birds don't speak")
```

LSP

- **Ensures Polymorphism Works Correctly:**
 - The client code (`animal_sound`) can interact with any `Animal` object without needing to know the exact subclass.
 - Simplifies the design and allows the system to be extended with new animals seamlessly.
- **Prevents Fragile Code:**
 - If a subclass violates LSP, client code might break unexpectedly when interacting with that subclass, leading to fragile and hard-to-maintain code.
- **Supports Open-Closed Principle:**
 - LSP is closely tied to the **Open-Closed Principle (OCP)**.
 - By ensuring that subclasses respect the contract of the base class, you can add new functionality (new subclasses) without modifying existing client code.



Can a code obey OCP but violate LSP?

Breaking LSP w/o breaking OCP

```
# Base Class
class Animal:
    def speak(self):
        raise NotImplementedError("Subclasses must implement this method")

# Subclass 1: Dog
class Dog(Animal):
    def speak(self):
        return "Woof!"

# Subclass 2: Cat
class Cat(Animal):
    def speak(self):
        return "Meow!"

# Subclass 3: Fish (violates LSP)
class Fish(Animal):
    def speak(self): # Fish can't "speak" in the traditional sense
        raise ValueError("Fish cannot speak!")
```

Breaking LSP w/o breaking OCP

```
# Factory to create animals
class AnimalFactory:
    @staticmethod
    def create_animal(animal_type):
        if animal_type == "dog":
            return Dog()
        elif animal_type == "cat":
            return Cat()
        elif animal_type == "fish":
            return Fish()
        else:
            raise ValueError("Unknown animal type")

# Client code
def animal_sound(animal_type):
    animal = AnimalFactory.create_animal(animal_type) # Factory ensures OCP compliance
    return animal.speak()

# Usage
print(animal_sound("dog")) # Output: Woof!
print(animal_sound("cat")) # Output: Meow!
print(animal_sound("fish")) # Raises ValueError: Fish cannot speak!
```


Breaking LSP w/o breaking OCP (no Factory Pattern)

```
def animal_sound(animal: Animal):  
    return animal.speak()  
  
# Following OCP: New animals can be added without modifying `animal_sound`  
print(animal_sound(Dog())) # Output: Woof!  
print(animal_sound(Cat())) # Output: Meow!  
print(animal_sound(Fish())) # Raises ValueError: Fish cannot speak!
```

Factory Update with OCP

```
# Factory to create individual animals
```

```
class AnimalFactory:
```

```
    @staticmethod
```

```
    def create_animal(animal_type):
```

```
        if animal_type == "dog":
```

```
            return Dog()
```

```
        elif animal_type == "cat":
```

```
            return Cat()
```

```
        else:
```

```
            raise ValueError(
```

```
class Bird(Animal):
```

```
    def speak(self):
```

```
        return "Tweet!"
```

```
# Registering Bird without modifying the factory's core logic
```

```
AnimalFactory.register_animal("bird", Bird)
```

```
class AnimalFactory:
```

```
    _animal_map = {
```

```
        "dog": Dog,
```

```
        "cat": Cat,
```

```
    }
```

```
    @staticmethod
```

```
    def register_animal(animal_type, animal_class):
```

```
        AnimalFactory._animal_map[animal_type] = animal_class
```

```
    _animal_map:
```

```
    dict[str, type[Animal]] = {
```

```
        "dog": Dog, "cat": Cat, "bird": Bird
```